Asynchronous Program in JS

→ Asynchronous programing also JS to perform non-blocking operations

→ Instead of waiting long tasks

↳ • API calls • file reads

to complete, JS can continue executing other code while operation in the background.

- Better user experience
- Efficient resource utilization

Async/Await

→ using async/await provide more ~~better~~ synchronous way to handle asynchronous operations, particularly those involving promises.

async

→ placed before function to designate it as asynchronous operation.

→ An async always implicitly returns a Promise

await

→ can only be used inside an async function.

→ pauses the execution of async function until promise it's waiting for is settled.

→ if Promise resolves, await return resolved value.

→ if Promise rejects, await throws reject value.

```
async function fetchData() {
    const result = await fetch(.....);
    return result;
};
fetchData()
  .then(data => console.log(dat))
  .catch(error => console.error(error));
```

Error Handling try/catch

try block

→ contains code that is to be monitored for errors

→ If error occured, execution of try block stopped immediately, and control is transfered to catch block.

→ if no error, the catch is skipped with

complete execution of try block

catch block

→ only executed if error thrown in try block.

→ recieve error objects as parameter which has info about error

```
or));
async function fetchUserData(){
    try{
        const response = await fetch('api');
ed      const data = await response.json();
        return data;
    } catch(error){
o1      console.error('Failure', error);
        throw error;
    }
with }
}
```

Multiple Async Operation

Sequential Operations

```
async function sequentialOperation(){
    const user = await fetchUser();
    const posts = await fetchUserPosts(use
const comments = await fetchPostComment
                    (Post[0].id);
    return (user, posts, comments);
}
```

parallel Operations

```
async function paralelloperation(){
    const [users, post] = await
      Promise.all{[
          fetchUser(),
          fetchPost()
    }];
      return {users posts}}
```

Fetch API

→ modern, promise-based interface for making network requests

→ fetching resources from server

→ alternative of XMLHttpRequest

→ core of Fetch api is fetch method that take at lest one arguement which is URL of resource.

options: An optional objects to configure the request

(e.g; method, headers body for POST request)

GET/POST Request

Sending GET Request

→ GET requests are used to retrieve data from specified resource

```
const response = await fetch('api');
const data = await response.json();
```

Sending POST request

→ POST request used to send data to server to create or update a resource.

→ the data include request body

```
const response = await fetch('api' {
    {
        method : 'POST',
```

```
headers:{
  'content-Type': 'application/json',
},
  body: JSON.stringify({
    name: 'John DOE',
    email: 'jhon@example.com'
  })
});
```

⇒ **How does fetch() work under the hood?**

- fetch() creates HTTP request and returns a Promis that resolves the Response Object.
- It uses browser's networking capability to make actual

HTTP request. 7A

- The Promise resolves as soon as response headers are received not when the full response body is downloaded.